BLL-Dokumentation – Überarbeitungsversion

Moderne Webentwicklung am Beispiel einer alternativen Vertretungsplan-Anwendung

Zweck dieses Dokuments: Volltext der BLL mit allen fehlenden Quellenverweisen und inhaltlichen Erweiterungen, klar markiert zur Übertragung in Google Docs.

Legende der Markierungen:

▶ VERWEIS: Gelb = Fußnote/Quellenverweis hier einfügen

GRÜN = NEUER TEXTBLOCK, DER ERGÄNZT WERDEN MUSS

▶ Blau = Neue Quelle im Quellenverzeichnis ergänzen

Grau/kursiv = Bestehender Verweis (bereits vorhanden, nur zur Orientierung)

Neue Quellen (in Kapitel 11 ergänzen)

Krug, Steve: *Don't Make Me Think – A Common Sense Approach to Web Usability*. 3. Auflage, New Riders, 2014. (Usability-Prinzipien, nutzerorientierte Gestaltung, iteratives Interface-Design)

Spillner, Andreas / Linz, Tilo: *Basiswissen Softwaretest*. 6. Auflage, dpunkt.verlag, 2019. (Testmethodik, Teststufen, Testarten und Testprozess)

Nielsen, Jakob: 10 Usability Heuristics for User Interface Design. Nielsen Norman Group, 1994 (aktualisiert 2024), https://www.nngroup.com/articles/ten-usability-heuristics/

(abgerufen am 11.02.2026)

---

1 Kurzfassung

Der offizielle Vertretungsplan der Friedrich-Dessauer-Schule Limburg war für Schülerinnen und Schüler nur über WebUntis zugänglich – eine Plattform, die einen schulischen Account voraussetzt und keine gezielte Filterung nach eigenen Fächern ermöglicht. Ziel dieser Besonderen Lernleistung war es, eine personalisierte, öffentlich zugängliche Vertretungsplan-Webanwendung zu entwickeln, die diesen Informationszugang grundlegend verbessert.

Die Entwicklung verlief in drei Iterationen: Eine erste statische Version ohne Backend zeigte die Grenzen eines filterfreien Ansatzes. Eine zweite, auf Next.js basierende Version ohne Authentifizierung machte die Notwendigkeit einer sauberen Komponentenstruktur und API-Trennung deutlich. Die dritte und aktuelle Version wurde von Grund auf neu entwickelt und vereint Nutzerauthentifizierung, eine relationale Datenbank, einen cachegestützten API-Proxy und ein Delta-basiertes Push-Benachrichtigungssystem.

Die technische Realisierung erfolgte mit Next.js (App Router), NextAuth, Prisma (PostgreSQL) und dem Web-Push-Standard (VAPID). Die WebUntis-Schnittstelle wurde durch systematisches Reverse-Engineering erschlossen. Architektur, Datenmodell und Benachrichtigungslogik wurden eigenständig konzipiert. Das System ist produktiv im Einsatz; 134 automatisierte Testfälle sichern die Kernlogik ab.

✚ ERWEITERUNG – ERGEBNIS ERGÄNZEN

Die fünf definierten SMART-Ziele wurden in weiten Teilen erreicht: Die vier Kernziele (S1–S4) sind vollständig erfüllt, das fünfte Ziel zur Wartbarkeit ist weitgehend umgesetzt.

Die Arbeit ist dem Fach Informatik zuzuordnen und umfasst Konzeption, Implementierung und Dokumentation einer vollständigen Webanwendung.

---

2 Einleitung und Motivation

2.1 Ausgangssituation

Vertretungspläne sind im Schulalltag eine zentrale Informationsquelle. In vielen Schulen liegen diese Informationen zwar digital vor, sind für Lernende aber oft nur umständlich nutzbar: lange Tabellen, wenig mobile Optimierung und fehlende Filter für konkrete Klassen oder Fächer.

An der Friedrich-Dessauer-Schule Limburg war der Vertretungsplan als automatisch durchlaufende Ansicht in WebUntis verfügbar – ohne Suchfunktion und ohne Möglichkeit, gezielt nach der eigenen Klasse oder einem Fach zu filtern. Wer seinen Eintrag finden wollte, musste warten, bis er erschien.

2.2 Motivation des Projekts

Das Projekt fds-vertretungsplan adressiert genau diese Lücke. Ziel ist keine neue Datenquelle, sondern eine deutlich bessere Darstellung und Nutzbarkeit der vorhandenen Vertretungsdaten. Damit rückt die Software vom reinen Anzeigen langer Listen zu einem alltagsnahen Werkzeug für den Schulalltag.

2.3 Warum Web statt nativer App?

Die Entscheidung für eine Webanwendung ist fachlich sinnvoll:

- Keine Installation auf Endgeräten notwendig.
- Sofort nutzbar auf Smartphone, Tablet und Desktop.
- Zentrale Updates ohne App-Store-Prozess.

✚ ERWEITERUNG – PUSH-FÄHIGKEIT BEGRÜNDEN

Seit 2023 unterstützen alle großen Browser – einschließlich Safari auf iOS – Web-Push-Benachrichtigungen.

> ► VERWEIS: Vgl. W3C Push API, https://www.w3.org/TR/push-api/.

Damit entfällt der wichtigste funktionale Vorteil einer nativen App für dieses Projekt: Push-Notifications sind auch ohne App Store realisierbar.

2.4 Projektziele im BLL-Kontext

1. Vertretungen schnell und verständlich anzeigen.
2. Fehlerfälle robust behandeln.
3. Datenschutz und Sicherheit systematisch berücksichtigen.
4. Eine wartbare Struktur für spätere Erweiterungen schaffen.

2.5 Einordnung als Informatik-BLL

Dieses Projekt ist eine Informatik-BLL, weil es über eine reine Webseite hinausgeht. Im Mittelpunkt stehen Systementwurf, Algorithmen und technische Abwägungen – nicht visuelle Gestaltung.

2.6 Bedienung der Anwendung

Der typische Nutzungsablauf ist bewusst kurz gehalten:

1. Öffentliche Nutzer rufen die Startseite auf und können den Vertretungsplan direkt ansehen.
2. Wer persönliche Funktionen nutzen will, registriert sich und meldet sich an.
3. Danach führt die Anwendung zur Eingabe des persönlichen Stundenplans.
4. Im Dashboard werden passende Vertretungen, Filter und später auch Benachrichtigungen gebündelt angezeigt.

Damit ist die Bedienung im Alltag auf wenige Schritte reduziert: aufrufen, Datum wählen, Ergebnisse lesen und bei Bedarf den eigenen Stundenplan pflegen.

✚ ERWEITERUNG – SCREENSHOTS EINFÜGEN (2–3 ABBILDUNGEN)

Screenshot 1: Öffentliche Startseite – Zeigt die Vertretungsübersicht mit Datumsauswahl, Suchfeld und Kategorienfilter. *Beschreibung:* „Die Startseite ist ohne Login erreichbar. Datum wählen, filtern, Ergebnis lesen – drei Schritte."

Screenshot 2: Persönliches Dashboard – Zeigt das Dashboard eines angemeldeten Nutzers mit hervorgehobenen relevanten Vertretungen. *Beschreibung:* „Nach dem Login sieht der Nutzer nur die für seinen Stundenplan relevanten Vertretungen."

Screenshot 3: Push-Einstellungen – Zeigt die Einstellungsseite mit dem Push-Toggle. *Beschreibung:* „Push-Benachrichtigungen lassen sich mit einem Klick aktivieren. Der Browser fragt einmalig nach der Berechtigung."

2.7 Aufbau der Arbeit

Die Dokumentation gliedert sich in zehn Abschnitte. Nach dieser Einleitung beschreibt Kapitel 2 die Anforderungen aus Nutzersicht (Lastenheft). Kapitel 3 erläutert den technischen Lösungsansatz (Pflichtenheft), gefolgt von Projektverlauf und Zeitplanung in Kapitel 4. Kapitel 5 ordnet den verwendeten Tech Stack ein. Kapitel 6 zeigt die Systemstruktur anhand von UML-Diagrammen. Kapitel 7 dokumentiert Umsetzungserfahrungen und Lessons Learned. Kapitel 8 bewertet die Ergebnisse gegen die definierten SMART-Ziele. Kapitel 9 gibt einen Ausblick, und Kapitel 10 listet die verwendeten Quellen.

---

3 Informationsquellen und Anforderungen (Lastenheft)

3.1 Informationsquellen

Die Anforderungen wurden aus folgenden Quellen zusammengeführt:

- Schulalltag: Bedarf an schneller, mobiler Vertretungsinformation.
- Eigene Projektdokumentation und vorhandene Oberflächenstruktur.
- Rechtliche Anforderungen: Impressum und Datenschutz als Pflichtbestandteil.
- Qualitätsziele: Tests, Linting und robuste Fehlerdarstellung.

3.2 Zielgruppen und Beteiligte

- Primär: Schülerinnen, Schüler und Lehrkräfte.
- Sekundär: Projektbetreiber und Schule als organisatorischer Rahmen.
- Extern: WebUntis als datenlieferndes Fremdsystem.

3.3 Funktionale Anforderungen

Die Unterteilung in funktionale und nicht-funktionale Anforderungen folgt der Standardgliederung der Anforderungsanalyse. ✓ *Bestehende Fußnote: Vgl. Sommerville (2015), Kap. 4: Requirements Engineering.*

1. Tagesbezogene Anzeige von Vertretungen.
2. Such- und Filtermöglichkeiten für schnelle Orientierung.
3. Verständliche Darstellung bei leerem Plan, Teildaten oder Fehlern.
4. Registrierung, Login und persönlicher Stundenplan für angemeldete Nutzer.
5. Zugriff auf rechtliche Pflichtseiten.

3.4 Bedienanforderungen

Die Anwendung sollte nicht nur Informationen bereitstellen, sondern auch ohne Einweisung verständlich benutzbar sein.

> ► VERWEIS: Vgl. Krug (2014), Kap. 1–2: Eine Anwendung sollte selbsterklärend sein – Nutzer sollten nicht nachdenken müssen, um sie zu bedienen.

Daraus ergeben sich drei Bedienanforderungen:

1. Der Einstieg für Gäste muss sofort funktionieren, ohne dass zuerst ein Konto angelegt werden muss.
2. Der Wechsel von Registrierung zu Onboarding und danach ins Dashboard muss nachvollziehbar geführt werden.
3. Push-Notifications und persönliche Einstellungen müssen als Zusatznutzen erkennbar sein, ohne die Grundfunktion der Vertretungsanzeige zu überladen.

3.5 Nicht-funktionale Anforderungen

1. Kurze Antwortzeiten und stabile Darstellung.
2. Sicherheit im Browserbetrieb.
3. Wartbarkeit durch klare Modultrennung.

   > ► VERWEIS: Vgl. Sommerville (2015), Kap. 6: Modultrennung als Voraussetzung für Wartbarkeit und unabhängige Weiterentwicklung.

4. Nachvollziehbarkeit durch Tests und Dokumentation.

3.6 SMART-Ziele (S1 bis S5)

Die Zielformulierung orientiert sich am SMART-Prinzip: Jedes Ziel ist spezifisch, messbar, attraktiv, realistisch und terminiert. ✓*Bestehende Fußnote: Vgl. Balzert (2009), Kap. 4: Anforderungsermittlung und Zieldefinition.* Dadurch lässt sich in Kapitel 8 ein nachvollziehbarer Soll-Ist-Abgleich durchführen.

> *[SMART-Tabelle S1–S5 unverändert übernehmen]*

3.7 Zentrale Anwendungsfälle

Die folgenden Anwendungsfälle beschreiben die wichtigsten Abläufe aus Nutzersicht. Sie ergänzen das Use-Case-Diagramm in Kapitel 6 um konkrete Schritte.

> ► VERWEIS: Vgl. Kleuker (2018), Kap. 3: Anwendungsfälle als zentrales Mittel der Anforderungsbeschreibung.

UC1: Vertretungsplan abrufen (öffentlich). Ein Gast ruft die Startseite auf, wählt ein Datum und sieht die Vertretungen für diesen Tag. Suche und Kategorienfilter ermöglichen die Eingrenzung. Kein Login erforderlich.

UC2a: Registrierung. Ein neuer Nutzer gibt E-Mail-Adresse und Passwort ein. Die Anwendung prüft, ob die E-Mail-Domain für Registrierungen zugelassen ist. Bei Erfolg wird das Konto angelegt und der Nutzer ist eingeloggt.

UC2b: Onboarding (Stundenplan anlegen). Nach der Registrierung leitet die Anwendung automatisch zur Stundenplan-Eingabe. Der Nutzer wählt pro Stunde Wochentag, Fach und Lehrkraft. Häufig genutzte Kombinationen werden als Schnellauswahl angeboten. Nach dem Speichern leitet die Anwendung automatisch ins Dashboard weiter. Das Onboarding muss nur einmal durchlaufen werden.

UC3: Persönliche Vertretungen anzeigen. Ein angemeldeter Nutzer öffnet das Dashboard. Die Anwendung gleicht die Vertretungen des gewählten Tages mit dem hinterlegten Stundenplan ab und hebt relevante Treffer hervor.

UC4: Push-Benachrichtigungen aktivieren. Ein angemeldeter Nutzer aktiviert in den Einstellungen Push-Benachrichtigungen. Der Browser fragt die Berechtigung ab. Bei Zustimmung wird ein Web-Push-Abonnement angelegt. Ab sofort prüft der Hintergrundprozess alle 15 Minuten, ob sich relevante Vertretungen geändert haben, und sendet nur dann eine Nachricht.

UC5: Admin-Verwaltung (Lehrerkürzel und Nutzerrollen). Ein Administrator pflegt die Lehrerkürzel-Tabelle und kann Nutzerrollen ändern. Das Hochstufen eines Nutzers zur Admin-Rolle erfordert einen einfachen Klick. Das Herabstufen eines Admins ist durch einen Bestätigungsdialog gesichert: Der Administrator muss die E-Mail-Adresse des betroffenen Kontos eingeben, bevor die Änderung durchgeführt wird. Das System verhindert außerdem, den letzten vorhandenen Administrator herabzustufen.

---

4 Lösungsansatz und Pflichtenheft

4.1 Architekturprinzip

Die Architektur trennt vier Schichten: Die Benutzeroberfläche zeigt Daten an, eine Zwischenschicht verwaltet den lokalen Zustand, eine serverseitige API-Schicht bündelt Validierung und Fehlerbehandlung, und die externe Datenquelle (WebUntis) wird nur serverseitig kontaktiert. ✓*Bestehende Fußnote: Vgl. Sommerville (2015), Kap. 6: Architekturentwurf – Schichtenarchitekturen als Mittel zur Trennung von Zuständigkeiten.* Dadurch bleibt der Browser leichtgewichtig, während sensible Logik serverseitig gebündelt wird.

4.2 Hosting und Web-Bereitstellung

Die Anwendung ist als normale Next.js-Webanwendung ausgelegt. Das Projekt kann auf Vercel oder auf eigener Infrastruktur betrieben werden, solange eine öffentlich erreichbare HTTPS-URL vorhanden ist, da der Scheduler – ein externer Dienst der Firma Upstash – keine lokalen Adressen aufrufen kann.

Die Hosting-Entscheidung ist deshalb Teil des Pflichtenhefts: Ohne öffentliche URL, gesetzte Umgebungsvariablen und klaren Deploy-Prozess würden Dashboard, Registrierung und Push zwar lokal funktionieren, aber nicht als Webprodukt im echten Betrieb.

4.3 Nutzer anlegen, Onboarding und Admin-Setup

Der Nutzerfluss ist bewusst mehrstufig aufgebaut:

1. Neue Nutzer registrieren sich über die Registrierungsseite.
2. Ob ein Nutzer Admin-Rechte erhält, wird über die Umgebungskonfiguration festgelegt. So ist sichergestellt, dass niemand zufällig Admin-Rechte bekommt.
3. Der erste registrierte Nutzer sollte idealerweise ein Admin sein, da nur Admins die Lehrerkürzel auf der Admin-Seite pflegen können.
4. Nach erfolgreichem Login führt die Anwendung ins persönliche Onboarding oder – für Admins – ins Admin-Setup.
5. Erst nach hinterlegten Lehrkräften und einem eigenen Stundenplan ist das Dashboard der eigentliche Zielzustand.

Dieser mehrstufige Ablauf stellt sicher, dass Nutzer erst nach vollständiger Einrichtung ihres Profils das Dashboard erreichen. Das verhindert unvollständige Konten im Echtbetrieb.

4.4 Rollen und Berechtigungen

Die Anwendung kennt zwei Rollen: USER und ADMIN. Ein normaler Nutzer kann seinen Stundenplan pflegen, das Dashboard nutzen und Push-Benachrichtigungen aktivieren. Ein Administrator hat zusätzlich Zugriff auf die Lehrerkürzel-Verwaltung und die Nutzerverwaltung – er kann Rollen vergeben und Konten entfernen.

Die erste Admin-Zuweisung erfolgt über eine Umgebungsvariable: Nur E-Mail-Adressen, die dort hinterlegt sind, erhalten beim Registrieren Admin-Rechte. Dieser erste Administrator kann weitere Nutzer per Weboberfläche hochstufen. Das Hochstufen erfordert einen einfachen Klick; das Herabstufen eines Admins erfordert dagegen die Eingabe der E-Mail-Adresse des betroffenen Kontos in einem Bestätigungsdialog. Das System stellt außerdem sicher, dass immer mindestens ein Administrator vorhanden ist – der letzte Admin kann nicht herabgestuft werden.

✚ ERWEITERUNG – E-MAIL-DOMAIN-VALIDIERUNG

Zusätzlich ist die Registrierung auf bestimmte E-Mail-Domains eingeschränkt. Über die zentrale Konfiguration (AppSettings) legt der Administrator fest, welche Domains für die Anmeldung zugelassen sind. So wird sichergestellt, dass sich nur Personen mit einer passenden E-Mail-Adresse registrieren können – eine einfache, aber wirksame Zugangskontrolle ohne aufwendige Verifizierung.

4.5 Push-Notifications als Hintergrundprozess

Push-Benachrichtigungen wurden nicht als einfache „Sende immer alles"-Funktion umgesetzt. Stattdessen kombiniert die Anwendung mehrere Bausteine:

- persönlicher Stundenplan als Filtergrundlage,
- periodischer Dispatch über QStash, einen Funktions-Scheduler der Firma Upstash, dessen kostenloser Tarif für den Dauerbetrieb dieses Projekts ausreicht,
- serverseitiger Abgleich gegen relevante Vertretungen,
- Delta-Logik, damit nur neue oder geänderte Treffer versendet werden.

Ein externer Scheduler ist notwendig, weil eine Webanwendung von sich aus nicht aktiv wird – sie reagiert nur auf eingehende Anfragen. QStash übernimmt die Rolle eines

Weckers: Er sendet alle 15 Minuten eine Anfrage an die Anwendung und löst damit den Dispatch-Zyklus aus.

Der Hintergrundprozess prüft für jeden Nutzer mit aktivierten Benachrichtigungen, ob relevante Vertretungen vorliegen. Nur bei neuen oder geänderten Treffern wird über das VAPID-Protokoll (RFC 8292) eine Push-Nachricht versendet. ✓*Bestehende Fußnote: Vgl. RFC 8292: Voluntary Application Server Identification (VAPID) for Web Push.*

Die Push-Nachricht wird nicht direkt an die sichtbare Seite zugestellt, sondern an einen Service Worker. Ein Service Worker ist ein Hintergrundskript, das der Browser unabhängig von der geöffneten Seite ausführt. ✓*Bestehende Fußnote: Vgl. W3C Service Workers, https://www.w3.org/TR/service-workers/.* Er empfängt die eingehende Push-Nachricht, erzeugt die sichtbare Benachrichtigung und reagiert auf Klicks – etwa durch Öffnen des Dashboards. Dadurch funktionieren Benachrichtigungen auch dann, wenn die Anwendung gerade nicht geöffnet ist.

Die Logik funktioniert wie ein digitaler Fingerabdruck: Die Anwendung erstellt bei jedem Prüflauf einen Kurzwert über die relevanten Vertretungen des Nutzers. Stimmt er mit dem vorherigen überein, ist nichts Neues passiert – kein Push. Weicht er ab, wurde etwas geändert.

Wie funktioniert die Delta-Logik? Für jeden Nutzer wird bei jedem Dispatch-Zyklus ein Prüfwert (Fingerprint) über die aktuell relevanten Vertretungen berechnet und in der Datenbank gespeichert. Beim nächsten Zyklus wird der Fingerprint erneut berechnet und mit dem gespeicherten verglichen. Drei Ergebnisse sind möglich: *Senden* (neuer oder geänderter Fingerprint – es hat sich etwas geändert), *Überspringen* (Fingerprint unverändert – keine Neuigkeit) oder *Auflösen* (keine Treffer mehr – gespeicherter Zustand wird bereinigt). So werden Nutzer nur dann benachrichtigt, wenn sich für sie tatsächlich etwas geändert hat.

4.6 Warum nur alle 15 Minuten und nicht in Echtzeit?

Die 15-Minuten-Frequenz ist eine fachliche und technische Abwägung:

- WebUntis liefert hier keinen echten Event-Stream, sondern muss aktiv abgefragt werden.
- Ein sekundenweises Polling würde die Last auf das Fremdsystem und auf die eigene Anwendung unnötig erhöhen.
- Im Schulalltag reicht ein kurzer, regelmäßiger Aktualisierungsrhythmus aus; Sekunden-Echtzeit bringt kaum zusätzlichen Nutzen.

4.7 Technische Kernentscheidungen

Schedule-Matching. Der Abgleich zwischen persönlichem Stundenplan und Vertretungsliste vergleicht Wochentag, Stunde und Fach. Daraus ergibt sich ein Relevanzwert, der höher ist, wenn mehrere Kriterien übereinstimmen.

> ► VERWEIS: Vgl. Sommerville (2015), Kap. 4: Anforderungsanalyse – Ableitung von Verarbeitungsregeln aus Nutzungsszenarien.

Caching-Strategie. Die API nutzt einen serverseitigen Cache mit kurzer Gültigkeitsdauer. Bei Upstream-Fehlern werden zwischengespeicherte Daten weiterverwendet, damit Nutzer auch bei vorübergehenden Störungen stabile Ergebnisse erhalten. ✓*Bestehende Fußnote: Vgl. RFC 5861: HTTP Cache-Control Extensions for Stale Content (stale-while-revalidate, stale-if-error).*

✚ ERWEITERUNG – CACHING-STRATEGIE ERKLÄREN

Diese Strategie folgt dem Prinzip „stale data is better than no data": Veraltete, aber konsistente Daten sind im Schulkontext hilfreicher als eine Fehlermeldung. Die Anwendung unterscheidet dabei zwischen regulärem Cache-Ablauf (30 Sekunden) und einer verlängerten Notfall-Gültigkeit (30 Minuten), die nur bei Nichterreichbarkeit der Datenquelle greift.

4.8 Datenschutz und Sicherheitsarchitektur

Passwörter werden gehasht gespeichert, Sitzungen laufen über verschlüsselte Tokens mit sicheren Cookies. Der Browser wird durch Sicherheitsrichtlinien (Content Security Policy) geschützt, ✓*Bestehende Fußnote: Vgl. W3C Content Security Policy Level 3.* Login-Versuche sind ratenbegrenzt. Klarnamen oder Schülerdaten werden nicht erfasst. ✓*Bestehende Fußnote: Vgl. OWASP Top 10 (2021).*

✚ ERWEITERUNG – MIDDLEWARE ERKLÄREN

Die serverseitige Middleware prüft bei jedem Seitenaufruf, ob der Nutzer authentifiziert ist, und leitet nicht angemeldete Nutzer auf die Login-Seite um. Gleichzeitig setzt sie HTTP-Sicherheitsheader, die unter anderem verhindern, dass die Anwendung in einem fremden iFrame eingebettet oder durch Cross-Site-Scripting angegriffen wird.

► VERWEIS: Vgl. OWASP Top 10 (2021), A03: Injection – Content Security Policy als serverseitiger Schutzmechanismus gegen eingeschleusten Code.

Die Datenbank speichert Nutzerkonten, persönliche Stundenpläne, Lehrerzuordnungen und Push-Subscriptions.

Die gewählte Lösung erfüllt die zentralen Sollvorgaben des Pflichtenhefts: klare Nutzerführung, robuste API-Kontrolle, webtaugliches Hosting und eine Push-Architektur, die im Schulkontext sinnvoller ist als ein Echtzeitmodell.

4.9 Datenmodell

Die Datenbank (PostgreSQL, verwaltet über Prisma) speichert acht Entitäten:

► VERWEIS: Vgl. Balzert (2009), Kap. 6: Datenmodellierung – Entitäten, Attribute und Beziehungen als Grundlage relationaler Datenbanken.

> *[Entitäten-Tabelle unverändert übernehmen]*

Alle Nutzer-bezogenen Entitäten sind über Fremdschlüssel mit kaskadierendem Löschen verbunden: Wird ein Nutzerkonto entfernt, werden Stundenplan, Push-Abonnements und Benachrichtigungszustände automatisch bereinigt.

Zusammenhang der Entitäten im Push-Prozess. TimetableEntry ist die Grundlage für die Push-Logik: Der Dispatcher liest für jeden Nutzer seinen Stundenplan und gleicht ihn mit den Vertretungen des Tages ab. Nur Treffer aus diesem Abgleich fließen in die Benachrichtigungsentscheidung ein. Das Ergebnis – ein Prüfwert (Fingerprint) über die relevanten Treffer – wird in NotificationState gespeichert.

Warum NotificationState und NotificationFingerprint getrennt sind. NotificationState speichert genau eine Zeile pro Nutzer und Zieldatum: den zuletzt gesendeten Fingerprint. Er wird bei jedem Dispatch-Zyklus überschrieben und gelöscht, sobald keine Treffer mehr vorliegen. NotificationFingerprint hingegen wächst nur – jeder je gesendete Fingerprint wird dauerhaft protokolliert. Diese Trennung ist bewusst: Der State steuert, ob eine Benachrichtigung gesendet wird; der Fingerprint dokumentiert, was ein Nutzer zu welchem Zeitpunkt gesehen hat. Würde man beide in einer Tabelle zusammenfassen, ließe sich der Versandverlauf nicht mehr rekonstruieren.

Warum TimetablePreset neben TimetableEntry existiert. Ein TimetableEntry repräsentiert eine konkrete Unterrichtsstunde mit Zeitdaten (Wochentag, Stunde,

Wochenmodus). Ein TimetablePreset speichert nur die inhaltliche Kombination – Fach, Lehrkraft, Raum – ohne Zeitbezug. Er dient als Autocomplete-Gedächtnis im Stundenplan-Editor: Die am häufigsten genutzten Kombinationen erscheinen als Schnellauswahl. Beide Entitäten haben orthogonale Zwecke und lassen sich nicht sinnvoll zusammenlegen.

---

5 Projektmanagement und Zeitplanung

Das Projekt wurde iterativ entwickelt: ausprobieren, Probleme erkennen, nächsten Meilenstein ableiten. ✓*Bestehende Fußnote: Vgl. Sommerville (2015), Kap. 2: Softwareprozesse – iterative Entwicklung als Alternative zum Wasserfallmodell.* Ein formales Vorgehensmodell war bei einem Einzelprojekt nicht sinnvoll. Änderungen wurden in kleinen Schritten umgesetzt, automatisiert getestet und bei Erfolg übernommen.

5.1 Entwicklungsphasen und Vorgängerversionen

Das Thema wurde am 31. Januar 2025 formal eingereicht. Die eigentliche Implementierung begann im August 2025 – also mehrere Monate nach der Einreichung, nachdem der konzeptionelle Rahmen bereits feststand.

Version 1 – Statische Seite (August/September 2025)

Die erste Version war eine rein statische Website ohne Backend. Die Datumsauswahl erfolgte über URL-Parameter; die Vertretungsdaten wurden direkt im Browser von der WebUntis-Schnittstelle abgerufen. Diese Version zeigte schnell ihre Grenzen: Eine Filterung nach eigenem Stundenplan war technisch nicht möglich. Wer die Seite öffnete, sah alle Vertretungen auf einmal – ungefiltert, ohne Personalisierung. Die Bedienbarkeit entsprach damit kaum dem angestrebten Mehrwert gegenüber WebUntis selbst.

Version 2 – Next.js, öffentlich, ohne Authentifizierung (September–Oktober 2025)

Der nächste Ansatz setzte auf Next.js als Framework, verzichtete aber noch auf jede Form von Nutzerkonten oder Backend-Logik. Die gesamte Verarbeitung fand im Frontend statt. Mit wachsendem Funktionsumfang wurden die strukturellen Schwächen dieses Ansatzes deutlich: Es gab keine klare Komponentenstruktur, keine API-Schicht und keine Trennung von Daten und Darstellung. Neuen Code einzufügen bedeutete zunehmend, bestehende Teile zu destabilisieren. Der Aufwand für Pflege und Erweiterung überstieg den Nutzen einer inkrementellen Weiterentwicklung.

> ► VERWEIS: Vgl. Sommerville (2015), Kap. 9: Softwareevolution – die bewusste Entscheidung für einen Neustart statt inkrementeller Überarbeitung als fachlich begründete Maßnahme.

Version 3 – Vollständiger Neustart mit sauberer Architektur (ab Mitte November 2025)

Die Entscheidung für einen vollständigen Neustart fiel bewusst: Erst das Datenmodell definieren, dann die API-Schicht aufbauen, dann das Frontend. Authentifizierung, Rollen, Caching und Push-Benachrichtigungen wurden von Anfang an als Bestandteile des Systems eingeplant – nicht nachträglich angehängt. Diese Version ist die aktuelle, produktiv eingesetzte Anwendung.

Die Versionswechsel waren keine Fehlschläge, sondern Teil eines iterativen Erkenntnisprozesses: Jede Version klärte Anforderungen, die in der nächsten sauber umgesetzt werden konnten.

5.2 Arbeitsweise: Try-and-Error mit klaren Meilensteinen

> *[Meilenstein-Tabelle M1–M5 unverändert übernehmen]*

5.3 Gantt-Plan mit Meilensteinen

> *[Gantt-Diagramm unverändert übernehmen]*

Neue Anforderungen standen nicht am Anfang vollständig fest. Erst im Verlauf wurden aus der Grundidee ein webfähiger Nutzerfluss und später ein sinnvoller Push-Prozess.

5.4 Netzplan und kritischer Pfad

> *[Netzplan-Diagramm unverändert übernehmen]*

Der kritische Pfad verläuft vom MVP über API-Stabilität und Nutzerfluss bis zum Push-Dispatch. Push konnte erst sinnvoll werden, nachdem Hosting und API-Stabilität bereits gelöst waren.

5.5 Austausch und Rückmeldungen

Die Projektarbeit wurde durch regelmäßigen Austausch begleitet. Über das gesamte Projekt fanden laufende mündliche Rückmeldungen durch die betreuenden Lehrkräfte Alexander Rhode und Michael Knobl statt. Förmliche Gespräche mit schriftlichen Notizen konzentrierten sich auf die zweite Projekthälfte und betrafen konkret: die Kapitelstruktur

der Dokumentation, fehlende inhaltliche Punkte sowie inhaltliche Verbesserungen einzelner Abschnitte.

Darüber hinaus wurde die Anwendung von Mitschülerinnen und Mitschülern des Informatik-Kurses im Schulalltag getestet. Da sie unterschiedliche Smartphones verwendeten, traten Darstellungsprobleme auf mehreren Bildschirmgrößen auf: Texte und Tabellen wurden abgeschnitten, Layout-Elemente verschoben sich, und Schaltflächen lagen teilweise außerhalb des sichtbaren Bereichs. Diese Probleme wurden im Verlauf der Meilensteine M3 und M4 behoben. Zusätzlich entstanden aus den Rückmeldungen konkrete Feature-Requests, die im ursprünglichen Entwurf nicht vorgesehen waren: ein Dark Mode sowie eine verbesserte Datumsnavigation. Kleinere Detailwünsche wurden laufend eingearbeitet.

Einige technische Kernentscheidungen entstanden nicht durch externe Rückmeldungen, sondern aus eigener Initiative: die Delta-Logik für den Push-Prozess, die Erschließung der WebUntis-Schnittstelle per Browser-Netzwerkanalyse, das Rollenkonzept mit USER- und ADMIN-Berechtigungen sowie die Wahl von QStash als externem Scheduler. Diese Entscheidungen sind im Pflichtenheft (Kapitel 3) begründet.

5.6 Methodisches Vorgehen

Iteratives Vorgehen als bewusste Wahl

Für ein Einzelprojekt ohne festes Anforderungsdokument wäre ein klassisches Wasserfallmodell nicht geeignet gewesen.

> ► VERWEIS: Vgl. Balzert (2009), Kap. 3: Vorgehensmodelle – das Wasserfallmodell setzt stabile, vollständige Anforderungen voraus, die bei explorativen Projekten nicht gegeben sind.

Stattdessen wurde iterativ vorgegangen: Erst eine funktionierende Basis schaffen, dann schrittweise Funktionen ergänzen, testen und bei Bedarf neu bewerten. Die drei Entwicklungsversionen (V1–V3) sind Ausdruck dieses Prozesses, nicht Zeichen von Planlosigkeit – jede Version klärte Anforderungen, die in der nächsten sauber umgesetzt wurden.

Anforderungsermittlung aus dem Schulalltag

Die Anforderungen wurden nicht aus der Literatur abgeleitet, sondern aus konkreter Alltagserfahrung: der täglich unbefriedigenden Nutzung von WebUntis, Rückmeldungen von Mitschülerinnen und Mitschülern über fehlende Filterbarkeit sowie einer systematischen Analyse der Schwachstellen der bestehenden Anzeige. Dieser Ausgangspunkt machte die Anforderungen greifbar und prüfbar.

Entscheidungsvalidierung durch Tests und Rückmeldungen

Technische Entscheidungen wurden nicht nur intuitiv getroffen, sondern durch automatisierte Tests abgesichert (34 Testsuiten, 134 Testfälle).

> ► VERWEIS: Vgl. Spillner/Linz (2019), Kap. 2: Grundlagen des Testens – automatisierte Tests als Mittel zur frühen Fehlererkennung und Absicherung von Entwurfsentscheidungen.

Rückmeldungen aus dem Tester-Kreis flossen in konkrete Verbesserungen ein – etwa bei Darstellungsproblemen auf verschiedenen Bildschirmgrößen oder beim Wunsch nach einem Dark Mode. Förmliche Gespräche mit den betreuenden Lehrkräften dienten zur inhaltlichen Orientierung in der Dokumentationsphase.

Reverse-Engineering als Informatik-Methode

Die WebUntis-Schnittstelle bietet keine öffentliche API-Dokumentation. Der Zugang zu den Vertretungsdaten wurde durch systematisches Beobachten des Netzwerkverkehrs im Browser erschlossen: Welche Endpunkte werden aufgerufen? Mit welchen Parametern? Welche Antwortstrukturen kommen zurück? Dieses Vorgehen – Hypothese aufstellen, Parameter variieren, Antwort auswerten – entspricht einer klassischen informatischen Analysemethode und verlief ohne größere Hindernisse.

Eigenständige Entscheidungen vs. rückmeldungsgetriebene Änderungen

Das Projekt unterscheidet zwischen zwei Kategorien von Entscheidungen: Solche, die aus eigener fachlicher Überlegung entstanden (Delta-Logik, Rollenkonzept, QStash-Integration, Datenbankmodell), und solche, die auf externe Rückmeldungen zurückgehen (Dark Mode, responsive Layout, Datumsnavigation). Erstere sind im Pflichtenheft begründet; letztere dokumentieren, dass das System im laufenden Betrieb auf reale Nutzung reagiert hat.

---

6 Tools und Tech Stack

6.1 Ziel der Toolauswahl

Der Stack wurde nicht nach „neuester Trend" gewählt, sondern nach Projektanforderung: schnelle Iteration, stabile Auslieferung und gute Wartbarkeit für ein Schulprojekt mit professionellem Anspruch. Dabei spielte auch Vorerfahrung eine Rolle – Next.js und React waren aus früheren Arbeiten bereits vertraut, ebenso wie Upstash als Anbieter für serverlose Hintergrunddienste.

6.2 Warum Framework statt reinem HTML/CSS (kurz)

Ein Framework wie Next.js ist sinnvoller als reines HTML, weil Routing, Datenabruf, Authentifizierung und Sicherheit einheitlich gelöst werden müssen. Ohne Framework müssten diese Querschnittsthemen einzeln und fehleranfällig zusammengesetzt werden.

> ► VERWEIS: Vgl. Sommerville (2015), Kap. 16: Komponentenbasierte Softwareentwicklung – Frameworks als Mittel zur Wiederverwendung bewährter Architekturmuster.

6.3 Kernstack

- Next.js (App Router) für Frontend und interne API im selben Projekt.
- React + TypeScript für klar typisierte Komponentenlogik.
- Tailwind + Komponentenlayer für konsistente UI.
- Vitest + Testing Library für schnelle Regressionstests.

  > ► VERWEIS: Vgl. Spillner/Linz (2019), Kap. 5: Testautomatisierung – automatisierte Regressionstests als Voraussetzung für sichere inkrementelle Weiterentwicklung.

- ESLint für frühe Qualitätskontrolle.

---

7 UML-Diagramme und Abläufe

Die Diagramme zeigen das Projekt aus drei Perspektiven: Nutzung, Struktur und Laufzeit.
✓ Bestehende Fußnote: Vgl. Kleuker (2018), Kap. 3-5: UML-Notation für Anwendungsfall-, Klassen- und Objektdiagramme.

7.1 Use-Case-Diagramm

Das Use-Case-Diagramm zeigt die Nutzerperspektive mit den externen Akteuren WebUntis und PostHog.

> *[Use-Case-Diagramm unverändert übernehmen]*

7.2 Klassendiagramm

Das Klassendiagramm zeigt die wichtigsten Module und vor allem ihre Kommunikation. Im Mittelpunkt stehen drei Wege:

1. Die Seite bzw. das Dashboard initiiert Nutzeraktionen.
2. Der Hook kapselt Client-Zustand, Cache und Datenabruf.
3. API-Routen und Services sprechen mit WebUntis, Prisma oder dem Push-System.

Die Schichten sind klar getrennt: UI-Komponenten sprechen nie direkt mit externen Systemen, sondern über Hooks, API-Routen und abgegrenzte Services.

> *[Klassendiagramm unverändert übernehmen]*

7.3 Objektdiagramm

Das Objektdiagramm zeigt eine konkrete Laufzeitsicht mit realen Instanzen. Anders als das Klassendiagramm beschreibt es keine allgemeinen Modultypen, sondern einen beispielhaften Betriebszustand.

> *[Objektdiagramm unverändert übernehmen]*

Neben dem sichtbaren Dashboard existiert im Hintergrund ein zweiter Laufpfad über QStash und die Push-API.

7.4 Sequenzdiagramm: Push-Dispatch

Das Sequenzdiagramm zeigt den zentralen Hintergrundprozess: den Push-Dispatch-Zyklus, der alle 15 Minuten prüft, ob Nutzer über geänderte Vertretungen benachrichtigt werden müssen.

> *[Sequenzdiagramm unverändert übernehmen]*

Der Ablauf zeigt die drei möglichen Ergebnisse der Delta-Logik: Senden bei neuem oder geändertem Fingerprint, Überspringen bei unverändertem Stand und Auflösen, wenn keine relevanten Vertretungen mehr vorliegen.

---

8 Umsetzung und Schwierigkeiten

8.1 Lesson Learned 1: Gute Bedienung entsteht nicht von allein

Am Anfang lag der Fokus stark auf der reinen Vertretungsanzeige. Erst im Verlauf der Entwicklung wurde deutlich, dass ein brauchbares Produkt auch einen geführten Nutzerfluss braucht: Registrierung, Admin-Ersteinrichtung, Onboarding und Dashboard mussten konzeptionell aufeinander abgestimmt werden. Ohne diesen Ablauf wäre zwar funktionierender Code vorhanden gewesen, aber kein in sich schlüssiger Arbeitsablauf für echte Nutzerinnen und Nutzer.

> ► VERWEIS: Vgl. Krug (2014), Kap. 3: Der Nutzer erwartet eine klare visuelle Hierarchie und selbsterklärende Abläufe – nicht nur korrekte Funktionen.

Der wesentliche Erkenntnisgewinn war: Produktqualität erschöpft sich nicht in korrekter Datenverarbeitung. Weiterleitungslogik, Rollenkonzept und klare Einstiegspunkte sind gleichwertige Bestandteile der eigentlichen Systemfunktion – nicht nachgelagerte Kosmetik.

8.2 Lesson Learned 2: Echtzeit klingt besser, ist hier aber nicht sinnvoll

Die 15-Minuten-Frequenz ist ein bewusstes Ergebnis aus technischem Rahmen und praktischem Nutzen. WebUntis liefert keinen Event-Stream, häufigeres Polling würde Last und Fehlerpotenzial erhöhen, und im Schulalltag zählt Verlässlichkeit mehr als Sekunden-Echtzeit.

8.3 Lesson Learned 3: Push-Notifications brauchen Zustandslogik

Es reicht nicht, bei jeder Änderung blind eine Nachricht zu verschicken. Erst mit persönlichem Stundenplan, Delta-Logik und einem geplanten Dispatcher wurde daraus eine Benachrichtigung, die im Alltag wirklich hilfreich ist. Die Einsicht: Benachrichtigungen sind nur dann gut, wenn sie sparsam, relevant und technisch kontrolliert ausgeliefert werden.

8.4 Lesson Learned 4: Hosting zwingt zu sauberen Schnittstellen

Sobald das Projekt nicht nur lokal, sondern als öffentliche Webanwendung gedacht wurde, veränderte sich der Anspruch. Eine externe URL, gesetzte Secrets, geschützte Routen und ein reproduzierbarer Deploy-Prozess wurden zu Pflichtpunkten. Gerade für QStash und Push zeigte sich, dass eine lokale Entwicklungsumgebung noch kein Beweis für echten Webbetrieb ist. Hinzu kam, dass der Scheduler-Dienst QStash ausschließlich HTTPS-Endpunkte akzeptiert. Diese Anforderung erzwang ein SSL-Zertifikat, brachte aber gleichzeitig den Vorteil, dass sämtlicher Datenverkehr zwischen Nutzer und Anwendung verschlüsselt übertragen wird.

8.5 Lesson Learned 5: Oberfläche gestalten braucht inhaltliche Klarheit zuerst

Die Gestaltung der Benutzeroberfläche erwies sich als einer der aufwendigsten Teile des Projekts. Das Interface durchlief insgesamt drei Iterationen: von einer rein funktionalen ersten Fassung über eine grundlegend bedienbare zweite bis hin zu einer Oberfläche, die funktional, hilfreich und ansprechend ist. Die entscheidende Erkenntnis war dabei, dass Gestaltungsentscheidungen erst dann sinnvoll getroffen werden können, wenn der Funktionsumfang und die inhaltliche Struktur feststehen.

> ► VERWEIS: Vgl. Nielsen (1994): Die Heuristik „Match between system and the real world" beschreibt genau dieses Prinzip – Gestaltung folgt dem Verständnis der Nutzeraufgabe, nicht umgekehrt.

Wer zu früh an der Oberfläche arbeitet, überarbeitet sie zwangsläufig, sobald sich die Anforderungen konkretisieren.

8.6 Wo es nicht sofort funktioniert hat

Rückblickend gab es drei typische Stolperstellen, die sich durch alle Entwicklungsphasen zogen:

1. Der Produktumfang war anfangs zu breit gedacht und musste auf einen klaren Nutzerkern reduziert werden. Die ersten beiden Versionen (V1 und V2, beschrieben in Kapitel 3a) zeigen, wie dieser Klärungsprozess verlief.
2. Der Nutzerfluss zwischen Registrierung, Admin-Setup und Onboarding war erst nach mehreren Überarbeitungen wirklich in sich schlüssig – technische Korrektheit und tatsächliche Bedienbarkeit sind zwei verschiedene Dinge.

3. Push-Benachrichtigungen erforderten deutlich mehr Zustandslogik als anfangs erwartet: Ohne Delta-Vergleich, Fingerabdruck-Speicherung und geplanten Dispatcher hätten die Nutzer bei jeder Abfrage eine Nachricht erhalten – unabhängig davon, ob sich etwas verändert hatte.

Diese Punkte waren keine Rückschläge ohne Nutzen, sondern genau die Phasen, aus denen der größte Lerngewinn entstand.

8.7 Grenzen der Lösung

Kein Mehrmandantenbetrieb. Das System ist auf eine Schule (FDS Limburg) bezogen. Mehrere Schulen zu unterstützen würde bedeuten, für jede separate WebUntis-Zugangsdaten zu haben.

Keine native App. Web Push funktioniert auf Apple-Geräten (iPhone, iPad und Mac) als auch auf Desktop zuverlässig.

Kein formales DSGVO-Audit. Technische Maßnahmen (Hashing, HTTPS, CSP, minimale Datenhaltung) sind umgesetzt, ✓*Bestehende Fußnote: Vgl. OWASP Top 10 (2021): Die gewählten Maßnahmen adressieren u.a. Injection (A03), Security Misconfiguration (A05) und Identification and Authentication Failures (A07).* eine juristische Prüfung lag jedoch außerhalb des Projektumfangs.

8.8 Eigenleistung und Werkzeugeinsatz

Kein Framework liefert eine fertige Vertretungsplan-App. Die Eigenleistung liegt in Architektur, Stundenplanabgleich, Delta-basierter Benachrichtigungslogik und Caching – also genau den Teilen, die das Projekt von einer reinen Konfigurationsarbeit unterscheiden.

Die WebUntis-Schnittstelle wurde ohne externe Dokumentation durch eigene Netzwerkanalyse erschlossen. Das Datenbankmodell, die Rollenstruktur und die Dispatch-Logik wurden selbst konzipiert. KI-Werkzeuge (konkret: Claude) wurden punktuell eingesetzt, um Entwürfe für das Datenbankschema zu validieren und auf strukturelle Schwachstellen zu prüfen. Alle Architekturentscheidungen – welche Tabellen, welche Relationen, welche Indexierung – wurden eigenständig getroffen; die KI diente dabei als kritisches Gegenüber, nicht als Entscheidungsinstanz. Implementierung, Tests und Dokumentation entstanden vollständig in eigener Arbeit.

---

9 Ergebnisbewertung (Ziele erfüllt?)

Die Bewertung orientiert sich an den SMART-Zielen aus Kapitel 02. ✓*Bestehende Fußnote: Vgl. Balzert (2009), Kap. 4: SMART-Kriterien als Grundlage messbarer Zielformulierung und systematischer Ergebnisbewertung.*

9.1 Soll-Ist-Abgleich zu den SMART-Zielen

> *[Soll-Ist-Tabelle S1–S5 unverändert übernehmen]*

Die Kernziele S1 bis S4 sind erreicht, S5 ist in weiten Teilen umgesetzt.

9.2 Testnachweis

Die Qualitätssicherung stützt sich auf automatisierte Tests mit Vitest und React Testing Library.

> ► VERWEIS: Vgl. Kleuker (2018), Kap. 7: Testverfahren – die Kombination aus Unit- und Integrationstests deckt sowohl isolierte Bausteine als auch deren Zusammenspiel ab.

Insgesamt umfasst das Projekt 34 Testdateien mit 134 Testfällen in drei Kategorien.

> *[Testfall-Tabelle unverändert übernehmen]*

Alle automatisierten Tests laufen über `npm run test:run` und sind in die Entwicklungsroutine integriert. Zusätzlich wurde die Anwendung manuell im Echtbetrieb mit mehr als sieben Nutzern getestet: Login-Ablauf, Push auf verschiedenen Geräten (Chrome, Safari, Arc) und Admin-Verwaltung wurden dabei fortlaufend überprüft.

---

10 Ausblick

10.1 Weiterentwicklung

Der nächste logische Schritt ist eine Testgruppe mit mehr als sieben aktiv nutzenden Personen, um belastbares Feedback zu Bedienbarkeit und Alltagstauglichkeit zu sammeln. Daraus ließen sich gezielt weitere Verbesserungen ableiten:

1. feinere Personalisierung für Klassen, Kurse oder Lehrkräfte,
2. klarere Priorisierung wichtiger Änderungen,
3. Testabdeckung für Randfälle in Push, Auth und Fehlerbehandlung erweitern,
4. Admin-Funktionen für Nutzerverwaltung und Datenpflege vereinfachen,
5. systematischere Beobachtung von Push-Erfolg und Fehlerquoten.

Solange das nicht der Fall ist, bleibt der 15-Minuten-Dispatcher die vernünftigere Lösung.

10.2 Fazit

Das Projekt hat sein zentrales Ziel erreicht: eine Webanwendung, die Vertretungsdaten der Friedrich-Dessauer-Schule schnell, gefiltert und personalisiert bereitstellt. Die vier Kernziele (S1–S4) sind vollständig erfüllt, das fünfte Ziel (S5, Wartbarkeit) ist in weiten Teilen umgesetzt.

Die technisch anspruchsvollsten Eigenleistungen liegen im Stundenplan-Abgleich, der Delta-basierten Push-Logik und der mehrstufigen Caching-Strategie. Diese Bausteine unterscheiden das Projekt von einer reinen Konfigurationsarbeit und zeigen, dass auch ein Schulprojekt belastbare Architekturentscheidungen erfordert.

Gleichzeitig bleiben Grenzen bestehen: Die Anwendung ist auf eine Schule bezogen, hat kein formales DSGVO-Audit durchlaufen und wird als Webanwendung statt als native App betrieben. Diese Einschränkungen waren von Anfang an bekannt und bewusst in Kauf genommen.

Rückblickend war der größte Lerngewinn nicht die Implementierung einzelner Features, sondern das Zusammenspiel von Nutzerführung, API-Robustheit, Hintergrundprozessen und Hosting zu einem funktionierenden Gesamtprodukt.

---

11 Quellen

11.1 Projektinterne Quellen

- Repository fds-vertretungsplan (Code, Tests, Konfiguration).
- WebUntis-Schnittstelle (FDS): erschlossen durch Browser-Netzwerkanalyse des öffentlich erreichbaren Vertretungsplans; keine offizielle API-Dokumentation vorhanden.

11.2 Technische Dokumentationen

- Next.js: https://nextjs.org/docs (abgerufen am 11.02.2026)
- React: https://react.dev (abgerufen am 11.02.2026)
- TypeScript: https://www.typescriptlang.org/docs/ (abgerufen am 11.02.2026)
- Vitest: https://vitest.dev/ (abgerufen am 11.02.2026)
- Sphinx: https://www.sphinx-doc.org/ (abgerufen am 11.02.2026)
- PlantUML: https://plantuml.com/ (abgerufen am 11.02.2026)
- PostHog: https://posthog.com/docs (abgerufen am 11.02.2026)
- Prisma: https://www.prisma.io/docs (abgerufen am 11.02.2026)
- Upstash QStash: https://upstash.com/docs/qstash (abgerufen am 11.02.2026)

11.3 Webstandards und Protokolle

- RFC 8292: Voluntary Application Server Identification (VAPID) for Web Push, https://datatracker.ietf.org/doc/html/rfc8292
- RFC 5861: HTTP Cache-Control Extensions for Stale Content (stale-while-revalidate), https://datatracker.ietf.org/doc/html/rfc5861
- W3C Push API, https://www.w3.org/TR/push-api/ (abgerufen am 11.02.2026)
- W3C Service Workers, https://www.w3.org/TR/service-workers/ (abgerufen am 11.02.2026)

11.4 Sicherheitsstandards

- OWASP Top 10 – 2021, https://owasp.org/Top10/ (abgerufen am 11.02.2026)
- W3C Content Security Policy Level 3, https://www.w3.org/TR/CSP3/ (abgerufen am 11.02.2026)

11.5 Fachliteratur Software Engineering

- Sommerville, Ian: *Software Engineering*. 10. Auflage, Pearson, 2015. (Anforderungsanalyse, Lastenheft-/Pflichtenheft-Struktur, Schichtenarchitektur)
- Balzert, Helmut: *Lehrbuch der Softwaretechnik: Basiskonzepte und Requirements Engineering*. 3. Auflage, Spektrum Akademischer Verlag, 2009. (Anforderungsermittlung, SMART-Methodik, Pflichtenheft-Abgrenzung)
- Kleuker, Stephan: *Grundkurs Software-Engineering mit UML*. 5. Auflage, Springer Vieweg, 2018. (UML-Notation, Anwendungsfall- und Klassenmodellierung, Teststrategien)

NEU: Krug, Steve: *Don't Make Me Think – A Common Sense Approach to Web Usability*. 3. Auflage, New Riders, 2014. (Usability-Prinzipien, nutzerorientierte Gestaltung)

NEU: Spillner, Andreas / Linz, Tilo: *Basiswissen Softwaretest*. 6. Auflage, dpunkt.verlag, 2019. (Testmethodik, Teststufen, Testprozess)

11.6 Webbasierte Fachreferenzen

NEU: Nielsen, Jakob: 10 Usability Heuristics for User Interface Design. Nielsen Norman Group, 1994 (aktualisiert 2024), https://www.nngroup.com/articles/ten-usability-heuristics/ (abgerufen am 11.02.2026)

11.7 Rechtliche und organisatorische Quellen

- Impressum und Datenschutzerklärung sind in der Anwendung enthalten.

Anhang: Übersicht aller Änderungen

A. Neue Verweise im Fließtext (15 Stück, zusätzlich zu 12 bestehenden)

Nr.	Kapitel	Stelle im Text	Quelle
1	2.3	Web-Push seit 2023 (Erweiterung)	W3C Push API
2	3.4	Selbsterklärende Bedienung	Krug (2014)
3	3.5	Wartbarkeit durch Modultrennung	Sommerville (2015)
4	3.7	Anwendungsfälle – Einleitung	Kleuker (2018)
5	4.7	Schedule-Matching	Sommerville (2015)
6	4.8	Middleware/Sicherheitsheader (Erweiterung)	OWASP Top 10
7	4.9	Datenmodell – Einleitung	Balzert (2009)
8	5.1	Version 2 – Neustart-Entscheidung	Sommerville (2015)
9	5.6	Wasserfallmodell-Vergleich	Balzert (2009)
10	5.6	Testabsicherung	Spillner/Linz (2019)
11	6.2	Framework-Entscheidung	Sommerville (2015)
12	6.3	Regressionstests	Spillner/Linz (2019)
13	8.1	Lesson Learned: Bedienung	Krug (2014)
14	8.5	Lesson Learned: Oberfläche	Nielsen (1994)
15	9.2	Testnachweis – Testverfahren	Kleuker (2018)

Gesamt nach Ergänzung: ~27 Verweise im Fließtext

B. Neue Textblöcke (Erweiterungen)

Nr.	Kapitel	Inhalt	Umfang
1	1	Ergebnis-Satz (SMART-Ziele) in Kurzfassung	1 Satz

2	2.3	Web-Push seit 2023 begründet „Web statt App"	2 Sätze
3	2.6	3 Screenshots mit Beschreibungen	~½ Seite
4	4.4	E-Mail-Domain-Validierung (AppSettings)	3 Sätze
5	4.7	Stale-while-revalidate erklärt	3 Sätze
6	4.8	Middleware-Funktion erklärt	3 Sätze

C. Neue Quellen im Verzeichnis

Nr.	Quelle	Verwendet in
1	Krug (2014): *Don't Make Me Think*	Kap. 3.4, 8.1
2	Spillner/Linz (2019): *Basiswissen Softwaretest*	Kap. 5.6, 6.3
3	Nielsen (1994): 10 Usability Heuristics	Kap. 8.5

D. Sonstige Erinnerungen

- **Deckblatt:** Abgabedatum steht auf 17. März 2026 – prüfen ob korrekt
- **Seitenumfang:** Aktuell ~28 Seiten – mit Lehrer besprechen (Skill-Zielkorridor: 15–20 Seiten)